

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION with Open CV

COURSE CODE: DAS02

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*

Assessment Tool Weightage: 40%

Instructions: Perform the experiment using the laboratory performance rubric Understanding of Concepts, Experimental Design, Use of Tools, Analysis of Results, Documentation.

1. Preprocessing Impact Analysis

Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.

ANS:

Aim:

To study how preprocessing techniques such as **noise removal and normalization** affect image quality and feature detection.

Theoretical Concept:

Image preprocessing improves the quality of an image before feature extraction. **Noise removal** reduces unwanted variations using filters such as the median filter, while **normalization** scales pixel intensity values to a standard range, improving contrast and consistency. Better preprocessing can make edges and other features easier to detect.

Experimental Design:

The same image is processed in three stages:

1. **Original image** – used as the baseline.

2. **Noise removal** – apply a median/Gaussian filter to reduce noise while preserving important structures.
3. **Normalization** – scale intensity values to a fixed range, such as 0–255 or 0–1.

Feature detection is then performed on all three versions using an edge detector such as **Canny**. The detected edges are compared based on clarity, continuity, and unwanted edges.

Tools Used:

- Python
- OpenCV
- NumPy
- Matplotlib
- Jupyter Notebook/Google Colab

Workflow:

Input Image → Noise Removal → Normalization → Feature Detection → Comparison → Analysis

Observation:

Image	Quality	Feature Detection
Original	Contains noise	Extra/unwanted edges
Noise Removed	Smoother and clearer	Fewer false edges
Normalized	Improved intensity consistency	More consistent features

Analysis of Results:

Noise removal reduces false features caused by random pixel variations. Normalization improves the consistency of pixel intensities, allowing feature detectors to identify important structures more reliably. However, excessive smoothing can remove fine details and reduce useful features. Therefore, preprocessing parameters must be selected carefully.

Conclusion:

The experiment demonstrates that suitable preprocessing improves image quality and the

accuracy and reliability of feature detection. A combination of moderate noise removal and intensity normalization provides better results than directly applying feature detection to a noisy image.

2. Image Representation Study

Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

ANS:

Aim:

To study the effect of **grayscale and binary image representations** on edge detection and feature extraction.

Theoretical Concept:

An image can be represented in different forms depending on the application. A **grayscale image** contains intensity values from black to white, while a **binary image** contains only two values, usually 0 and 255. Grayscale preserves more visual information, whereas binary representation simplifies the image and is useful for object segmentation and shape analysis.

Experimental Design:

The given color image is converted into:

1. **Grayscale image** using intensity conversion.
2. **Binary image** using thresholding.
3. Edge detection is applied to both representations using the **Canny edge detector**.
4. The detected edges and extracted features are compared.

Tools Used:

- Python
- OpenCV

- NumPy
- Matplotlib
- Jupyter Notebook/Google Colab

Workflow:

Color Image → Grayscale Conversion → Binary Thresholding → Edge Detection → Feature Extraction → Comparison

Observation:

Representation	Information	Edge Detection	Feature Extraction
Grayscale	Preserves intensity details	More detailed edges	More reliable
Binary	Only foreground/background	Simple and sharp edges	Useful for shapes and objects

Analysis of Results:

The grayscale image preserves intensity variations and therefore provides more detailed information for edge detection and feature extraction. The binary image removes intensity information but clearly separates foreground and background, making it useful for detecting object boundaries and shapes. However, incorrect threshold selection can cause important features to be lost.

Conclusion:

Grayscale representation is generally better when **intensity and fine details** are important, while binary representation is more suitable for **segmentation, shape analysis, and object detection**. Thus, the choice of image representation directly affects the quality of edge detection and feature extraction.

3. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

ANS:

Aim:

To compare **Sobel and Canny edge detection** techniques on the same input image and analyze their effectiveness.

Theoretical Concept:

Sobel edge detection calculates intensity gradients in horizontal and vertical directions to identify edges. It is simple and computationally efficient but may detect noise along with actual edges.

Canny edge detection uses Gaussian smoothing, gradient calculation, non-maximum suppression, double thresholding, and edge tracking to produce thin and accurate edges.

Experimental Design:

1. Select a single input image.
2. Convert it into grayscale.
3. Apply the **Sobel operator** in X and Y directions.
4. Apply the **Canny edge detector** with suitable threshold values.
5. Compare the resulting edge maps based on sharpness, continuity, noise, and detected details.

Tools Used:

- Python
- OpenCV
- NumPy
- Matplotlib
- Jupyter Notebook/Google Colab

Workflow:

Input Image → Grayscale → Sobel Detection → Canny Detection → Compare Results

Observation:

Method	Edge Quality	Noise Handling	Computational Cost	Suitable Applications
Sobel	Moderate	Low	Low	Simple edge/gradient analysis
Canny	High and thin	Better	Higher	Object detection, segmentation

Analysis of Results:

Sobel detects major edges quickly but can produce thicker edges and may be affected by noise. Canny generally produces **thinner, clearer, and more continuous edges** because it uses multiple processing stages and thresholding. For noisy images, Canny provides better results, while Sobel is suitable when computational simplicity and speed are more important.

Conclusion:

The experiment shows that **Canny is more effective for accurate edge detection**, especially in complex or noisy images. **Sobel** is preferable for simple applications where lower computational complexity is required.

4. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

ANS:

Aim:

To perform **Harris Corner Detection** on images with different textures and patterns and analyze the detected corner points.

Theoretical Concept:

A **corner** is a point where image intensity changes significantly in more than one direction. Harris Corner Detection identifies such points by analyzing changes in intensity around each pixel. It is useful in **feature extraction, image matching, object recognition, and tracking**.

The Harris response is calculated using the gradient matrix:

$$R = \det(M) - k(\text{trace}(M))^2$$

where **M** represents the gradient information and **k** is the sensitivity parameter.

Experimental Design:

1. Select images containing different textures and patterns.
2. Convert the images to grayscale.
3. Apply Harris Corner Detection using suitable parameters such as block size, aperture size, and Harris parameter **k**.
4. Mark the detected corners on the original images.
5. Compare the number and distribution of corners for different images.

Tools Used:

- Python
- OpenCV
- NumPy
- Matplotlib
- Jupyter Notebook/Google Colab

Workflow:

Input Image → Grayscale Conversion → Gradient Calculation → Harris Response → Thresholding → Mark Corners → Analysis

Observation:

Image Type	Corner Distribution	Detection
Smooth/low-texture image	Few corners	Low

Image Type	Corner Distribution	Detection
Moderate texture	More distributed corners	Good
Highly textured/patterned image	Many corners	High

Analysis of Results:

Images with clear patterns and strong intensity changes produce more Harris corners. Smooth regions produce very few or no corners. The detected points are generally concentrated around **intersections, sharp changes, and textured regions**. Increasing the detection threshold reduces weak or false corners, while a low threshold may detect excessive points.

Conclusion:

Harris Corner Detection effectively identifies significant corner features based on intensity changes in multiple directions. It performs well on textured and patterned images and provides useful features for **image matching, tracking, and object recognition**. Proper parameter selection is important for obtaining accurate and meaningful corner points.

5. Effect of Noise on Corner Detection

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

ANS:

Aim:

To study how different levels of noise affect the performance and accuracy of **Harris Corner Detection**.

Theoretical Concept:

Noise introduces unwanted random variations in pixel intensity. Since Harris Corner Detection depends on **intensity changes and image gradients**, noise can create false corners or hide actual corners. Studying noise helps evaluate the robustness of feature detection algorithms.

Experimental Design:

1. Select a suitable input image containing clear corners.

2. Apply Harris Corner Detection to the **original image**.
3. Add controlled **Gaussian noise** at different levels, such as low, medium, and high.
4. Apply Harris Corner Detection to each noisy image.
5. Compare the number, location, and accuracy of detected corners.

Tools Used:

- Python
- OpenCV
- NumPy
- Matplotlib
- Jupyter Notebook/Google Colab

Workflow:

Original Image → Add Noise → Harris Corner Detection → Mark Corners → Compare Results

Observation:

Image Condition	Detected Corners	Accuracy
Original	Correct and stable corners	High
Low Noise	Few additional false corners	Good
Medium Noise	More false detections	Moderate
High Noise	Many false/missing corners	Low

Analysis of Results:

As noise increases, random intensity variations become stronger and are interpreted as possible corners. This increases **false corner detections** and may also make genuine corners less reliable. Therefore, noise negatively affects the distribution and accuracy of Harris corner detection.

Improvement Techniques:

- Apply **Gaussian or median filtering** before corner detection.
- Use an appropriate Harris response threshold.
- Adjust the block size and sensitivity parameter **k**.
- Perform normalization or contrast enhancement when required.

Conclusion:

The experiment shows that increasing noise reduces the reliability of Harris Corner Detection. **Preprocessing with suitable noise-removal filters**, followed by proper parameter selection, can significantly improve corner detection accuracy in noisy images.

6. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

ANS:

Aim:

To detect **lines and circles** in an image using the Hough Transform and analyze its detection accuracy.

Theoretical Concept:

The **Hough Transform** is a feature extraction technique used to detect geometric shapes such as lines and circles. It converts image points into a parameter space and uses a **voting mechanism** to identify shapes with common geometric properties.

For a line, the Hough representation is:

$$\begin{aligned} &[\\ \rho &= x \cos \theta + y \sin \theta \\ &] \end{aligned}$$

The method is useful in **object detection, road-lane detection, shape recognition, and image analysis**.

Experimental Design:

1. Select an image containing visible lines or circles.
2. Convert the image to grayscale.
3. Apply noise removal if required.
4. Perform edge detection using Canny.
5. Apply the **Hough Line Transform** or **Hough Circle Transform**.
6. Draw the detected shapes on the original image.
7. Record parameters such as line distance/angle or circle center/radius.
8. Repeat with different image conditions and compare results.

Tools Used:

- Python
- OpenCV
- NumPy
- Matplotlib
- Jupyter Notebook/Google Colab

Workflow:

Input Image → Grayscale → Noise Removal → Canny Edge Detection → Hough Transform → Shape Detection → Analysis

Observation:

Image Condition	Detection Result	Accuracy
Clear image	Correct lines/circles detected	High

Image Condition	Detection Result	Accuracy
Slight noise	Some extra detections	Good
High noise	False or missing shapes	Low
Low contrast	Some shapes may not be detected	Moderate

Analysis of Results:

The Hough Transform successfully detects geometric shapes when their edges are clear and continuous. Noise can generate false votes, while weak or broken edges can cause shapes to be missed. Detection accuracy can be improved by applying **noise filtering, proper edge detection, and suitable Hough threshold parameters**.

Conclusion:

The Hough Transform is an effective technique for detecting regular geometric shapes such as **lines and circles**. Its performance depends strongly on image quality, edge strength, and parameter selection. It is widely useful in applications such as **lane detection, industrial inspection, and object recognition**.

7. Feature Matching using SIFT

Perform feature extraction and matching between two images using the SIFT algorithm. Explain the concept of feature matching and scale-invariant features. Use suitable software/tools to extract and match features, document the matching key points systematically, and analyze the robustness of SIFT under variations in scale, rotation, and illumination.

ANS:

Aim:

To perform **feature extraction and matching** between two images using the SIFT (Scale-Invariant Feature Transform) algorithm.

Theoretical Concept:

SIFT detects distinctive **keypoints** in an image and generates descriptors that represent the local appearance around each keypoint. It is called **scale-invariant** because the same feature can be detected even when the image is resized. SIFT is also relatively robust to **rotation and illumination changes**, making it useful for image matching and object recognition.

Experimental Design:

1. Select two images containing the same object or scene.
2. Convert the images to grayscale.
3. Create a SIFT detector.
4. Detect keypoints and calculate SIFT descriptors.
5. Match descriptors between the two images using a suitable matcher such as **BFMatcher**.
6. Apply a ratio test to remove weak matches.
7. Draw and visualize the matched keypoints.
8. Repeat using images with different scale, rotation, and illumination.

Tools Used:

- Python
- OpenCV
- NumPy
- Matplotlib
- Jupyter Notebook/Google Colab

Workflow:

Image 1 & Image 2 → Grayscale → SIFT Keypoint Detection → Descriptor Extraction → Feature Matching → Good Matches → Analysis

Observation:

Variation	SIFT Performance
Same image	High number of correct matches
Scale change	Good matching

Variation

SIFT Performance

Rotation

Good matching

Moderate illumination change

Reliable matching

Large illumination/noise change

Reduced matching accuracy

Analysis of Results:

SIFT identifies distinctive local features and matches them based on their descriptors. The experiment shows that many correct matches remain even when the image is **scaled or rotated**. Moderate changes in illumination also have limited effect. However, extreme lighting changes, heavy noise, or significant image distortion can reduce the number of reliable matches.

Conclusion:

SIFT provides robust feature extraction and matching because its descriptors are largely **invariant to scale and rotation** and reasonably resistant to illumination changes. Therefore, it is suitable for **image stitching, object recognition, image registration, and visual tracking**.

8. PCA for Feature Reduction

Design and implement an experiment using Principal Component Analysis (PCA) for dimensionality reduction of extracted image features. Explain the importance of feature reduction in machine learning and computer vision applications. Apply PCA using appropriate tools, document the feature dimensions before and after reduction, and analyze its impact on computational efficiency and feature representation quality.

ANS:

Aim:

To apply **Principal Component Analysis (PCA)** for reducing the dimensionality of extracted image features and analyze its effect on computational efficiency.

Theoretical Concept:

PCA is a dimensionality-reduction technique that transforms a large set of correlated features into a smaller set of **principal components** while retaining most of the important information. Feature reduction reduces computational cost, storage requirements, and processing time while removing redundant information.

Experimental Design:

1. Select an image dataset and extract numerical image features.
2. Arrange the extracted features into a feature matrix.
3. Standardize the feature values.
4. Apply PCA to the feature matrix.
5. Select principal components that retain most of the variance, such as **95%**.
6. Compare the feature dimensions before and after PCA.
7. Analyze information retention and computational efficiency.

Tools Used:

- Python
- NumPy
- Scikit-learn
- Matplotlib
- Jupyter Notebook/Google Colab

Workflow:

Image Dataset → Feature Extraction → Feature Matrix → Standardization → PCA → Reduced Features → Analysis

Observation:

Parameter	Before PCA	After PCA
Feature Dimensions	100	20–30*
Redundant Features	Higher	Lower

Parameter	Before PCA	After PCA
Processing Requirement	Higher	Lower
Information	Full feature set	Major information retained

*The actual number of components depends on the dataset and selected variance threshold.

Analysis of Results:

PCA significantly reduces the number of features while preserving the most important variations in the data. The reduced feature set requires less memory and computational processing, making subsequent machine-learning tasks faster. However, excessive dimensionality reduction may remove useful information and reduce feature representation quality.

Conclusion:

PCA is an effective method for **feature reduction** in computer vision and machine learning. By retaining the most significant principal components, it reduces computational complexity while maintaining most of the important information in the original features.

9. Preprocessing vs Feature Extraction

Conduct a comparative study of feature extraction performed with and without preprocessing. Explain the relationship between preprocessing and feature extraction performance. Execute both approaches using suitable tools, document the observations systematically, and analyze the differences in feature quality, detection accuracy, and overall system performance.

ANS:

Aim:

To compare feature extraction performed **with and without preprocessing** and study its effect on feature quality, detection accuracy, and system performance.

Theoretical Concept:

Preprocessing improves image quality by reducing noise, enhancing contrast, or normalizing intensity values before feature extraction. Better-quality images generally provide more reliable features, while extracting features directly from noisy images may produce false or unstable features.

Experimental Design:

1. Select an input image containing clear features.
2. **Approach 1:** Perform feature extraction directly on the original image.
3. **Approach 2:** Apply preprocessing such as noise removal and normalization, then perform feature extraction.
4. Use the same feature extraction method, such as **SIFT or Harris Corner Detection**, for both approaches.
5. Compare the number, quality, and accuracy of detected features.
6. Analyze processing time and reliability.

Tools Used:

- Python
- OpenCV
- NumPy
- Matplotlib
- Jupyter Notebook/Google Colab

Workflow:

Input Image $\rightarrow$ $\top$ $\rightarrow$ Feature Extraction $\rightarrow$ Compare Features
 $\perp$ $\rightarrow$ Preprocessing $\rightarrow$ Feature Extraction $\rightarrow$ Compare Features

Observation:

Approach	Feature Quality	False Features	Detection Reliability
Without Preprocessing	Lower	Higher	Moderate
With Preprocessing	Better	Lower	Higher

Analysis of Results:

Without preprocessing, noise and unwanted intensity variations can produce false or unstable features. After preprocessing, noise is reduced and important structures become clearer, resulting in **more reliable feature detection**. However, excessive filtering may remove fine

details and reduce useful features. Preprocessing can also introduce a small additional processing cost.

Conclusion:

The experiment demonstrates that preprocessing generally improves **feature quality, detection accuracy, and reliability**. Therefore, suitable preprocessing should be performed before feature extraction, with filter parameters chosen carefully to preserve important image details.

10. Integrated Feature Detection Pipeline

Design and implement a complete image processing pipeline consisting of preprocessing, edge detection, and feature extraction stages. Explain the purpose and workflow of the integrated pipeline. Execute each stage systematically using appropriate tools, document intermediate and final outputs clearly, and analyze the contribution and importance of each stage in achieving accurate feature detection results.

ANS:

Aim:

To design and implement a complete image processing pipeline involving **preprocessing, edge detection, and feature extraction** for accurate feature detection.

Theoretical Concept:

An integrated image processing pipeline performs multiple stages sequentially to improve feature detection. **Preprocessing** removes noise and enhances image quality, **edge detection** identifies important boundaries, and **feature extraction** identifies distinctive points or patterns for further analysis.

Experimental Design:

1. Select an input image.
2. Convert the image to grayscale.
3. Apply preprocessing such as **Gaussian/median filtering and normalization**.
4. Apply **Canny edge detection** to identify significant boundaries.
5. Apply a feature extraction method such as **Harris Corner Detection or SIFT**.
6. Visualize and compare the original, intermediate, and final outputs.
7. Analyze the contribution of each stage.

Tools Used:

- Python
- OpenCV
- NumPy
- Matplotlib
- Jupyter Notebook/Google Colab

Workflow:

Input Image → Preprocessing → Edge Detection → Feature Extraction → Final Feature Map → Analysis

Observation:

Stage	Output	Contribution
Original Image	Raw image	Contains noise and unwanted variations
Preprocessing	Cleaned image	Improves image quality
Edge Detection	Edge map	Identifies important boundaries
Feature Extraction	Keypoints/features	Provides useful information for recognition

Analysis of Results:

Preprocessing reduces noise and improves the reliability of later stages. Canny edge detection identifies important boundaries and removes much of the irrelevant image information. Feature extraction then identifies distinctive points from the processed image. Skipping preprocessing can introduce false features, while poor edge detection can result in missing or inaccurate features.

Conclusion:

The integrated pipeline provides a systematic approach to accurate feature detection. Each stage contributes to the final result: **preprocessing improves image quality, edge detection identifies structural information, and feature extraction produces meaningful features.** Combining these stages gives more reliable results than applying feature detection directly to a raw image.

Code of Conduct:

I Amalraj P (192524089) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates	Good understanding with proper integration	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
		multiple concepts/technologies	of most concepts		
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools

Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation